

Computer Vision

План лекции

1. Введение
2. История
3. Представление изображения
4. Классические методы

Введение

Что это?

Опр. Компьютерное зрение (computer vision) - это научное направление в области ИИ и связанные с ним технологии

- получения изображений объектов реального мира
- обработки изображений
- извлечения информации из изображений

для решения прикладных задач без участия (полного или частичного) человека.

Объект CV: изображения, видео поток.



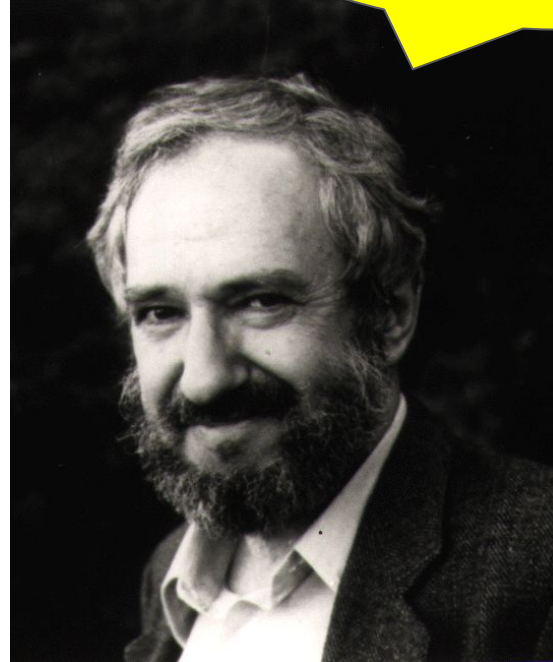
source: https://www.freepik.com/free-photo/enjoying-cycling_5534982.htm

Сложности CV

1966 Summer Vision Project (MIT)

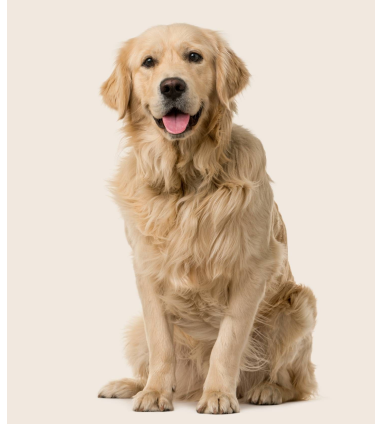
- отделять объекты от фона
- описывать фон и объекты
- идентифицировать, что изображено
- использовать повседневные объекты с различными цветами, шрифтами, текстурами, формой

“The summer vision project is an attempt to use our summer workers... in the construction of a significant part of a visual system” (1966)



Сложности CV

Разные позы объектов (object pose)



sources:

<https://www.petmd.com/dog/breeds/golden-retriever>

<https://manypets.com/uk/breeds/dogs/golden-retriever/>

<https://heronscrossing.vet/articles/the-upside-and-downsides-of-owning-a-golden-retriever/>

Сложности CV

Разнообразие объектов одного класса (intra-class variation)



sources:

https://en.wikipedia.org/wiki/Papillon_dog

https://en.wikipedia.org/wiki/Great_Dane

<https://www.britannica.com/animal/Siberian-husky>

Сложности CV

Окклюзия (occlusion)



Сложности CV

Шумный фон (cluttered background)



source: <https://pledgecare.org/stop-dog-chewing-house/>

Сложности CV

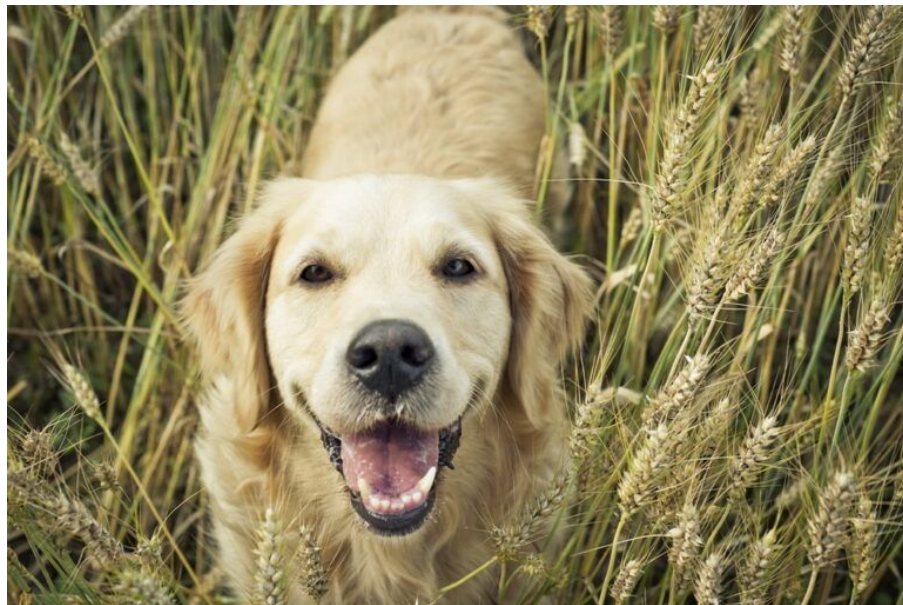
Освещение (illumination)



source: https://commons.wikimedia.org/wiki/File:Dog_in_the_forest.JPG

Сложности CV

Ракурс (viewpoint)



sources:

<https://www.greatpetcare.com/dog-breeds/golden-retriever/>

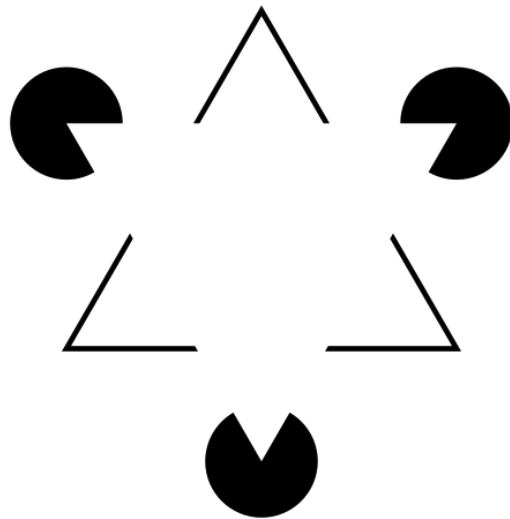
https://www.reddit.com/r/goldenretrievers/comments/auffn9/luna_looks_like_she_just_opened_the_front_facing/

Сложности CV

Сложность: хотим воспроизводить именно человеческое зрение

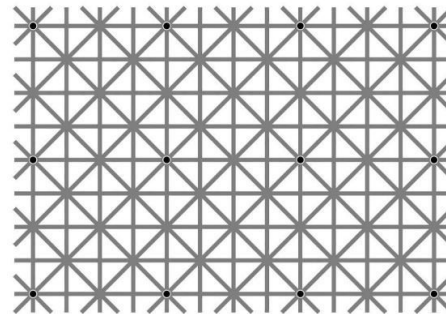
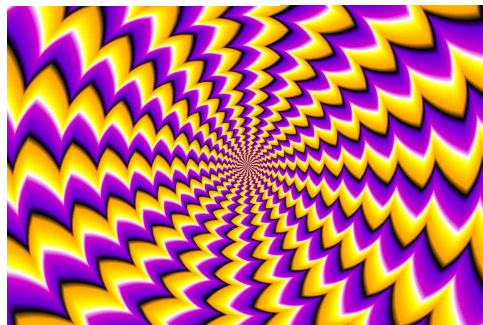
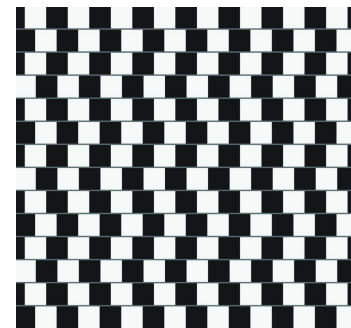
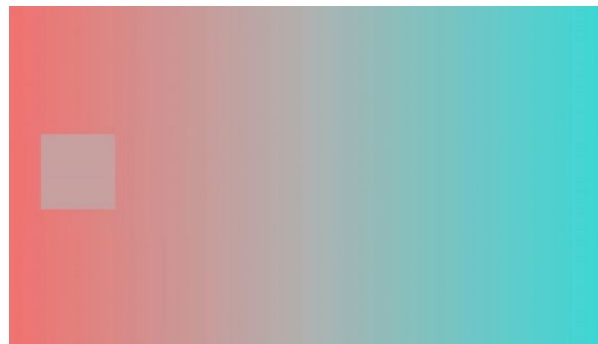
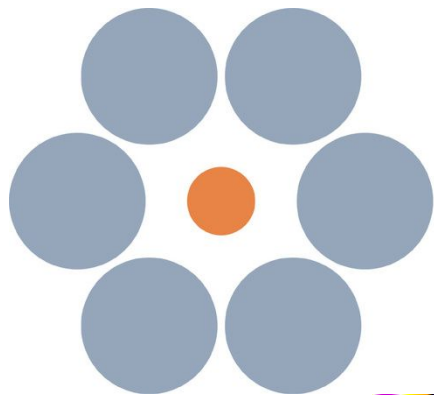
Сложности CV

Люди видят то, чего нет:



Сложности CV

Нас вообще легко обмануть:



Сложности CV

Разные люди видят разное:



Сложности CV

Восприятие зависит от предыдущего опыта:



Задачи компьютерного зрения.

- задачи распознавания
- задачи генерации
- комбинированные с другими модальностями

Задачи распознавания

- **ОСНОВНЫЕ**

- **классификация (classification)**
- локализация (localization)
- обнаружение объектов (object detection)
- сегментация (segmentation)

- **дополнительные**

- Оценка положения (pose estimation)
- Оценка глубины (depth estimation)
- ...

cat



dog



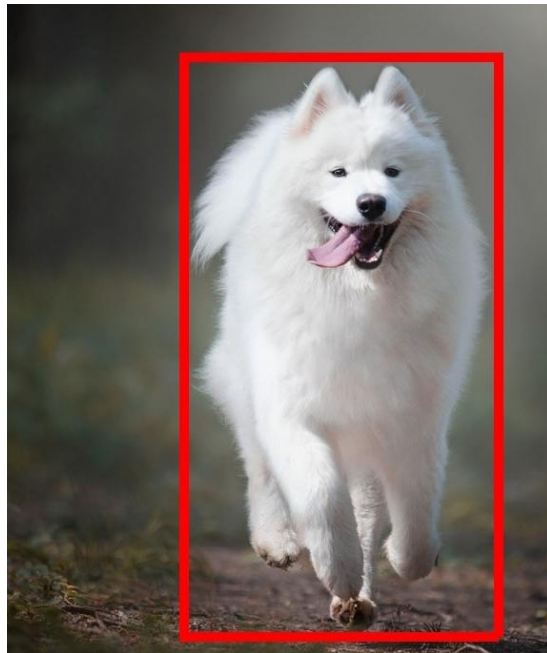
Задачи распознавания

- **ОСНОВНЫЕ**

- классификация (classification)
- **локализация (localization)**
- обнаружение объектов (object detection)
- сегментация (segmentation)

- **ДОПОЛНИТЕЛЬНЫЕ**

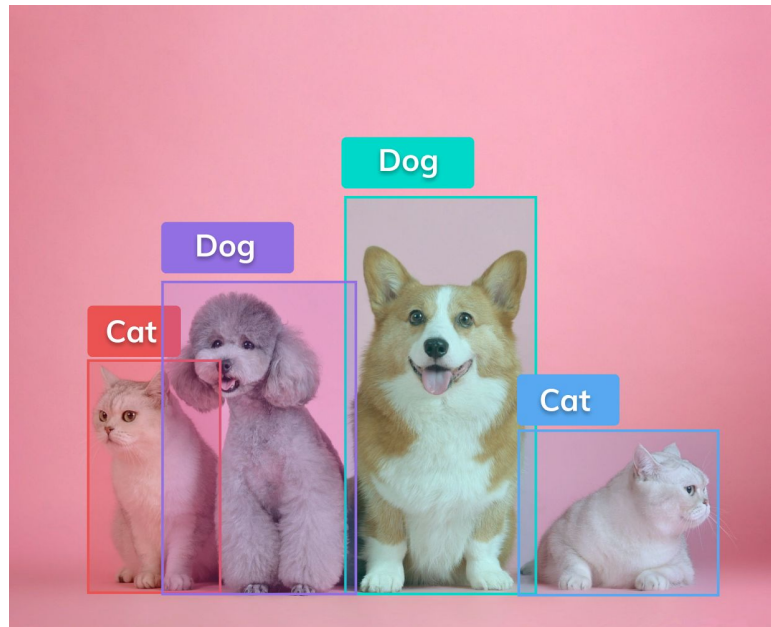
- Оценка положения (pose estimation)
- Оценка глубины (depth estimation)
- ...



source: <https://www.madpaws.com.au/wp-content/uploads/2021/03/Dog-Breeds-Samoyed.jpg>

Задачи распознавания

- **ОСНОВНЫЕ**
 - классификация (classification)
 - локализация (localization)
 - **обнаружение объектов (object detection)**
 - сегментация (segmentation)
- **ДОПОЛНИТЕЛЬНЫЕ**
 - Оценка положения (pose estimation)
 - Оценка глубины (depth estimation)
 - ...



source: <https://www.v7labs.com/blog/object-detection-guide>

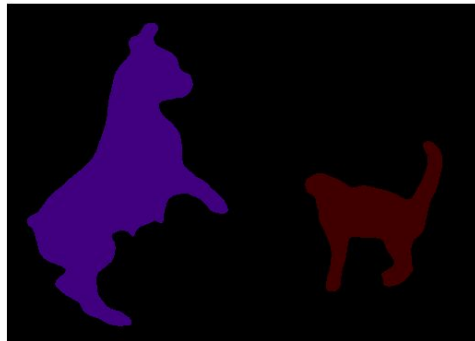
Задачи распознавания

- **ОСНОВНЫЕ**

- классификация (classification)
- локализация (localization)
- обнаружение объектов (object detection)
- **сегментация (segmentation)**

- **ДОПОЛНИТЕЛЬНЫЕ**

- Оценка положения (pose estimation)
- Оценка глубины (depth estimation)
- ...



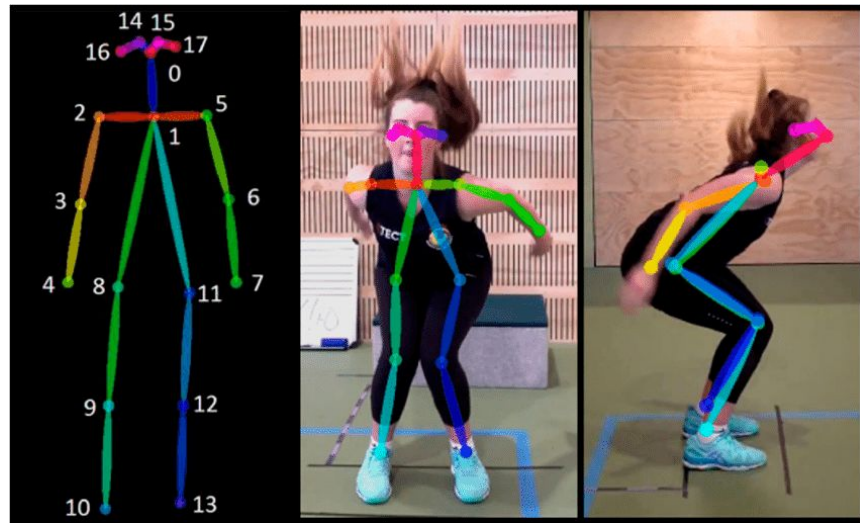
Задачи распознавания

- ОСНОВНЫЕ

- классификация (classification)
- локализация (localization)
- обнаружение объектов (object detection)
- сегментация (segmentation)

- ДОПОЛНИТЕЛЬНЫЕ

- **Оценка положения (pose estimation)**
- Оценка глубины (depth estimation)
- ...

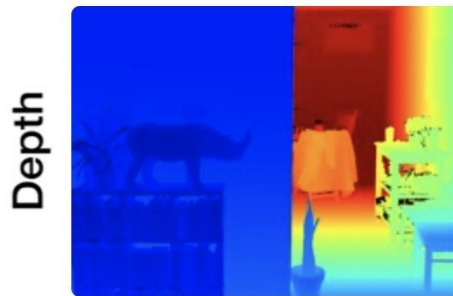
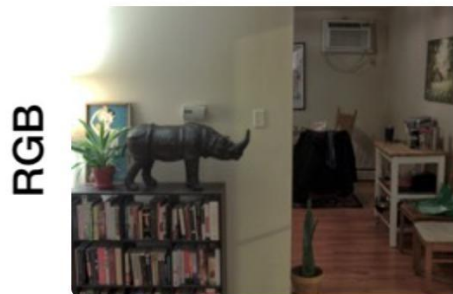


source:

https://www.researchgate.net/figure/OpenPoses-COCO-18-points-model-keypoint-positions-left-image-16-and-example-of-a_fig3_338905462

Задачи распознавания

- **ОСНОВНЫЕ**
 - классификация (classification)
 - локализация (localization)
 - обнаружение объектов (object detection)
 - сегментация (segmentation)
- **дополнительные**
 - Оценка положения (pose estimation)
 - **оценка глубины (depth estimation)**
 - ...



source: <https://arxiv.org/pdf/1908.00463.pdf>

Задачи генерации

- генерация изображения (generation)
- условная генерация изображения (conditional generation)
- перенос стиля (style transfer)
- вписывание части изображения (image inpainting)
- дорисовка изображения (image outpainting)



source: <https://paperswithcode.com/task/face-generation>

Задачи генерации

- генерация изображения (generation)
- условная генерация изображения (conditional generation)
- перенос стиля (style transfer)
- вписывание части изображения (image inpainting)
- дорисовка изображения (image outpainting)



monarch butterfly



goldfinch



daisy



redshank



grey whale

Задачи генерации

- генерация изображения (generation)
- условная генерация изображения (conditional generation)
- перенос стиля (style transfer)
- вписывание части изображения (image inpainting)
- дорисовка изображения (image outpainting)



Content Image

+



Style Image

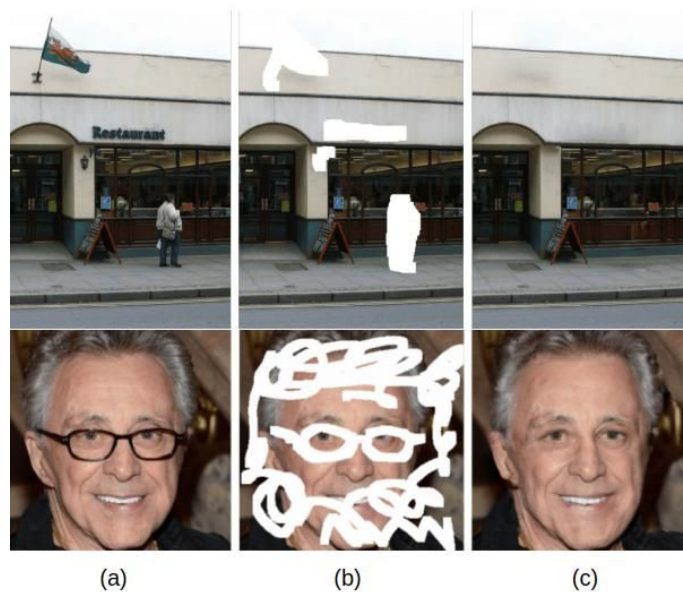
=



Neural Style Transfer

Задачи генерации

- генерация изображения (generation)
 - условная генерация изображения (conditional generation)
 - перенос стиля (style transfer)
 - **вписывание части изображения (image inpainting)**
 - дорисовка изображения (image outpainting)
- 



source: https://neerc.ifmo.ru/wiki/index.php?title=%D0%A4%D0%B0%D0%B9%D0%BB:Inpainting_sample.jpg

Задачи генерации

- генерация изображения (generation)
- условная генерация изображения (conditional generation)
- перенос стиля (style transfer)
- вписывание части изображения (image inpainting)
- дорисовка изображения (image outpainting)



source: <https://openai.com/blog/dall-e-introducing-outpainting>

Комбинированные с NLP

- генерация изображения по текстовому описанию (text-to-image)
- генерация текстового описания по изображению (image-to-text)
- ответ на вопросы по изображению (VQA)

Комбинированные с NLP

- генерация изображения по текстовому описанию (text-to-image)
- генерация текстового описания по изображению (image-to-text)
- ответ на вопросы по изображению (VQA)

Комбинированные с NLP

- генерация изображения по текстовому описанию (text-to-image)
- генерация текстового описания по изображению (image-to-text)
- ответ на вопросы по изображению (VQA)

Про данные

Домены данных

СТОКОВЫЕ картинки



фотографии на смартфон



ID vs OOD

Компоненты задачи в классическом ML:

- пространство объектов X
- множество ответов Y
- $P(x, y)$ - истинное распределение данных на $X \times Y$
- функция риска R
- обучающая выборка $D_{\text{train}} = \{x_i, y_i\}_{i=1..n_1} \sim P(x, y)$
- тестовая выборка $D_{\text{test}} = \{x_i, y_i\}_{i=1..n_2} \sim P(x, y)$

Допущение классического ML: на проде будут $D_{\text{prod}} \sim P \Rightarrow R_{\text{prod}} \approx ER \approx R_{\text{test}}$

Реальность: на проде данные $D_{\text{prod}} \sim Q \neq P$ – **сдвиг распределения (distribution shift)**

Важно: часто Q не известно

Источники сдвига

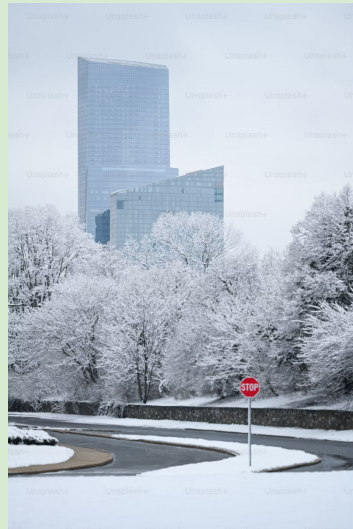
Низкоуровневый сдвиг: объекты те же, их представление меняется

Пример: изменение погодных условий

обучение



прод



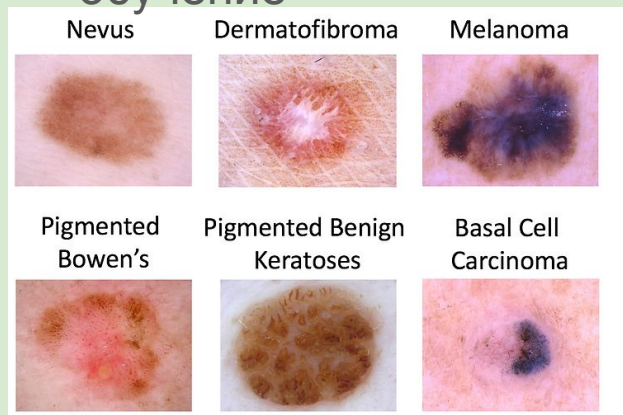
Источники сдвига

Семантический сдвиг: объекты нового класса

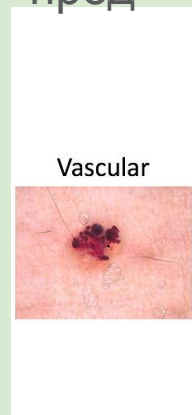
Пример:

- вы обучили модель распознавать 6 типов родинок
- пользователь по незнанию скормил ей иной, неподдерживаемый тип

обучение



прод



Выборки

При построении модели используются 3 выборки:

1. трейн (train, обучающая) - для обучения параметров
2. валид (valid, валидационная) - для настройки гиперпараметров
3. тест (test, тестовая) - для оценки обобщающей способности

Data leaks

Data leak - протечка данных между 2 выборками.

Почему плохо: data leak приводит к недостоверной оценке работы модели.

- ужасно: трейн+валид в тест -> недостоверная оценка итоговой модели
- плохо: трейн в валид -> неоптимальный выбор гиперпараметров

Супер-короткая история CV

История

- 1826 - первая фотография «Вид из окна в Ле Гра»
- 1888 - первое видео
- 1957 - первый компьютер: Манчестерский Марк I
- 1966 - Summer Vision Project
- 2001 - метод Виолы-Джонса
- 2012 - AlexNet
- 2017 - Attention is All You Need
- GPT-1



source: https://en.wikipedia.org/wiki/View_from_the_Window_at_Le_Gras

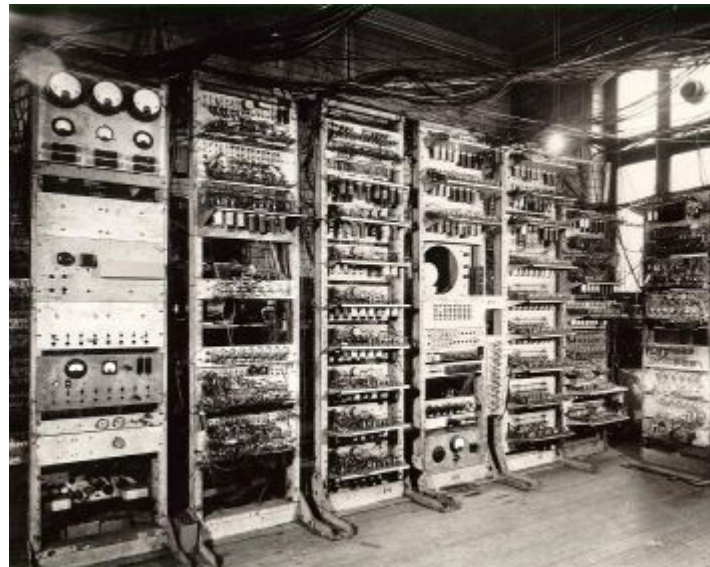
История

- 1826 - первая фотография «Вид из окна в Ле Гра»
- 1888 - первое видео
- 1957 - первый компьютер: Манчестерский Марк I
- 1966 - Summer Vision Project
- 2001 - метод Виолы-Джонса
- 2012 - AlexNet
- 2017 - Attention is All You Need



История

- 1826 - первая фотография «Вид из окна в Ле Гра»
- 1888 - первое видео
- 1957 - первый компьютер: Манчестерский Марк I
- 1966 - Summer Vision Project
- 2001 - метод Виолы-Джонса
- 2012 - AlexNet
- 2017 - Attention is All You Need

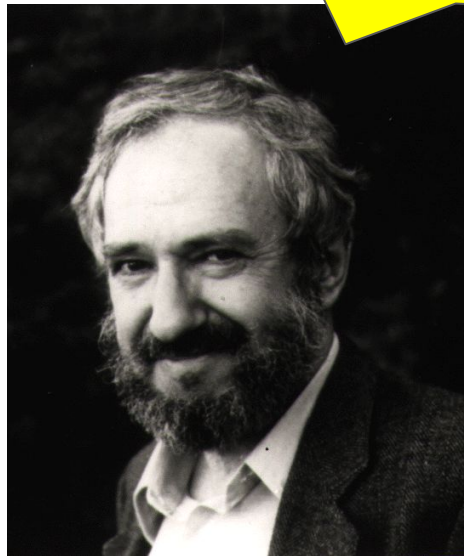


source: https://en.wikipedia.org/wiki/Manchester_Mark_1

История

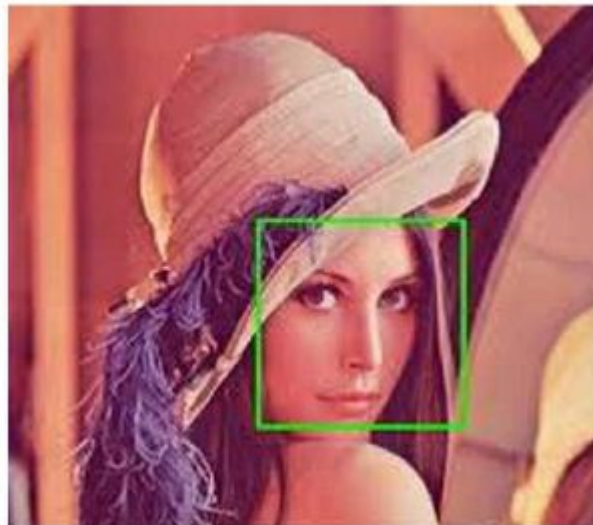
- 1826 - первая фотография «Вид из окна в Ле Гра»
- 1888 - первое видео
- 1957 - первый компьютер: Манчестерский Марк I
- 1966 - Summer Vision Project
- 2001 - метод Виолы-Джонса
- 2012 - AlexNet
- 2017 - Attention is All You Need

“The summer vision project is an attempt to use our summer workers... in the construction of a significant part of a visual system” (1966)



История

- 1826 - первая фотография «Вид из окна в Ле Гра»
- 1888 - первое видео
- 1957 - первый компьютер: Манчестерский Марк I
- 1966 - Summer Vision Project
- 2001 - метод Виолы-Джонса
- 2012 - AlexNet
- 2017 - Attention is All You Need

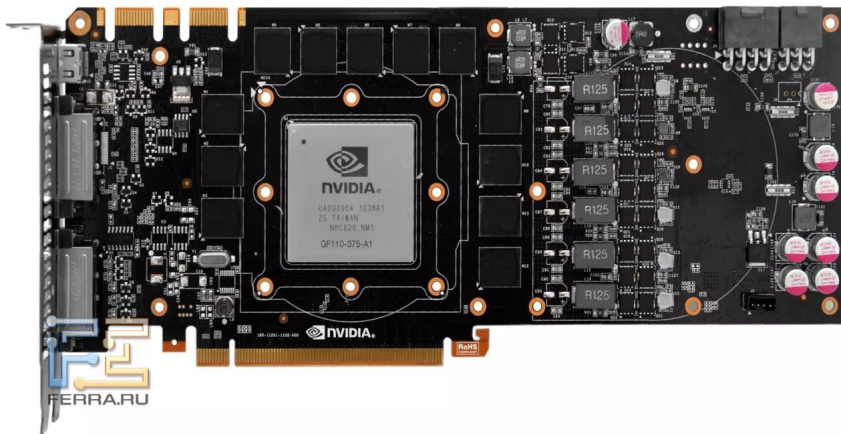


source:

https://www.researchgate.net/publication/282195030_OpenCV_WebCam_Applications_in_an_Arduino-based_Rover

История

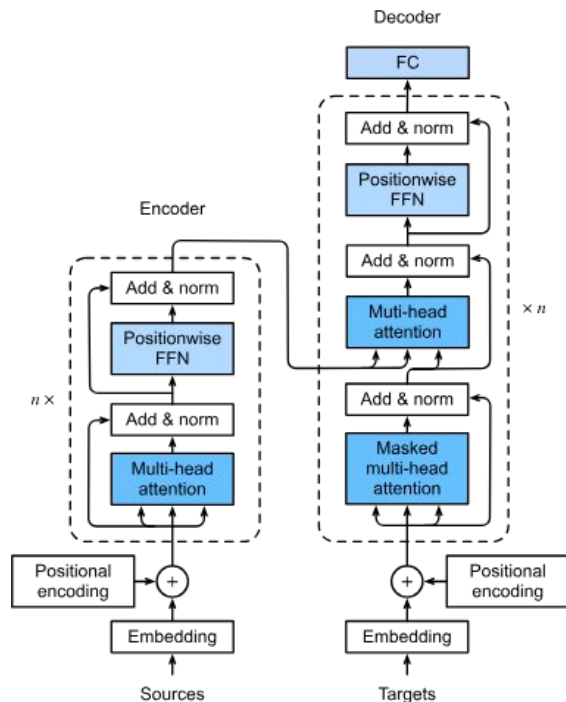
- 1826 - первая фотография «Вид из окна в Ле Гра»
- 1888 - первое видео
- 1957 - первый компьютер: Манчестерский Марк I
- 1966 - Summer Vision Project
- 2001 - метод Виолы-Джонса
- 2012 - AlexNet
- 2017 - Attention is All You Need



source: <https://www.ferra.ru/review/computers/Gainward-GTX-580-3-Gb-Phantom-3-videocard.htm>

История

- 1826 - первая фотография «Вид из окна в Ле Гра»
- 1888 - первое видео
- 1957 - первый компьютер: Манчестерский Марк I
- 1966 - Summer Vision Project
- 2001 - метод Виолы-Джонса
- 2012 - AlexNet
- 2017 - Attention is All You Need



Представление изображения

Зрение человека

Свет - электромагнитное излучение, воспринимаемое глазом человека.

Цвет - психологическое свойство зрения человека, возникающее при наблюдении объектов и света, а не физические свойства объектов и света.

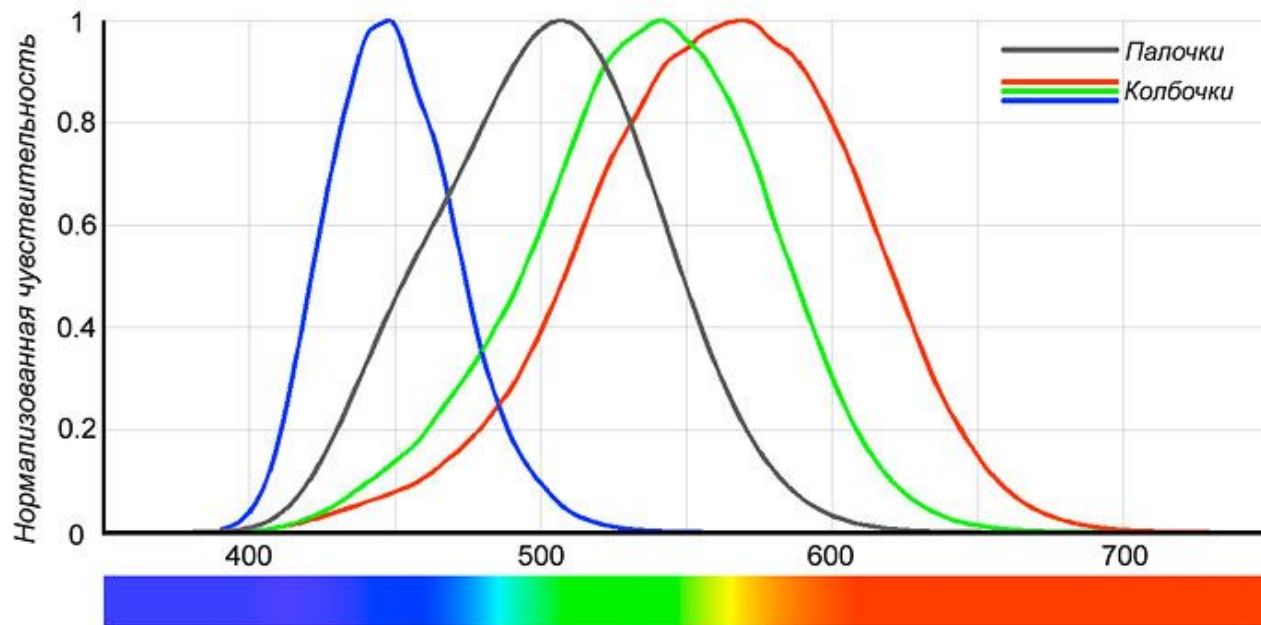
Процесс восприятия цвета

Этапы

1. источник света испускает электромагнитные волны
2. волны проходят через среду
3. опционально отражаются от какого-нибудь объекта
4. попадают в глаз
5. чувствительные клетки в сетчатке глаза реагируют на волны
6. сигнал от чувствительных клеток передается в затылочные доли мозга
7. возникают цветовые ощущения

Типы чувствительных клеток

- палочки
- колбочки
 - красные
 - зеленые
 - синие



source: <http://photomike.ru/notes/colors/>

Представление цвета

Цвет кодируется координатами в цветовом пространстве, например:

- RGB
- HSV / HSL
- L^*a^*b

RGB

Особенность: соответствие биологическим механизмам зрения

Три типа колбочек $\leftrightarrow$ 3 цветовых компоненты

Значения: (r,g,b)

- r - интенсивность красного цвета
- g - интенсивность зеленого цвета
- b - интенсивность синего цвета



HSV / HSL

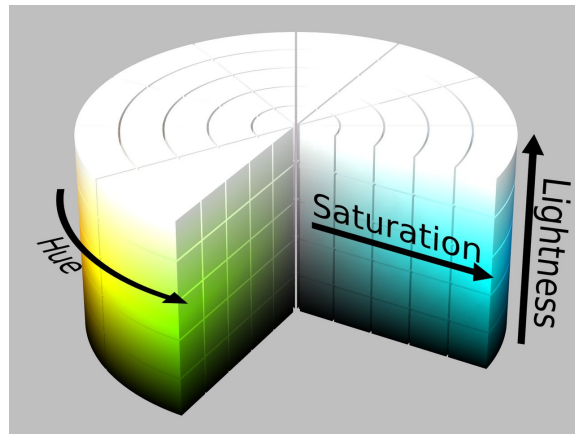
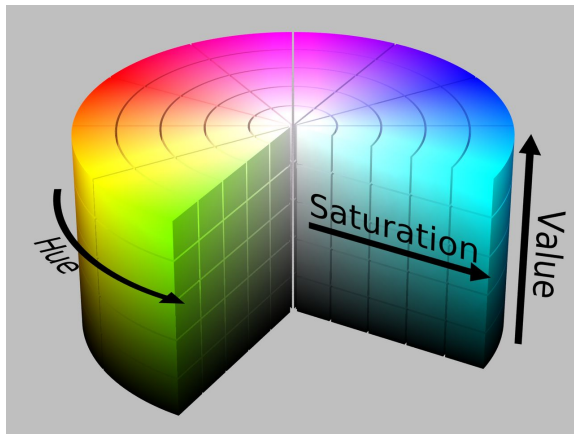
Особенность: интерпретируемость человеком

Моделируют:

- HSV - как выглядят цвета при разном освещении
- HSL - смешивание красок художниками

Значения: (h,s,v) / (h,s,l)

- hue (цветность)
- saturation (насыщенность)
- value/brightness (яркость)
- lightness (светлота)



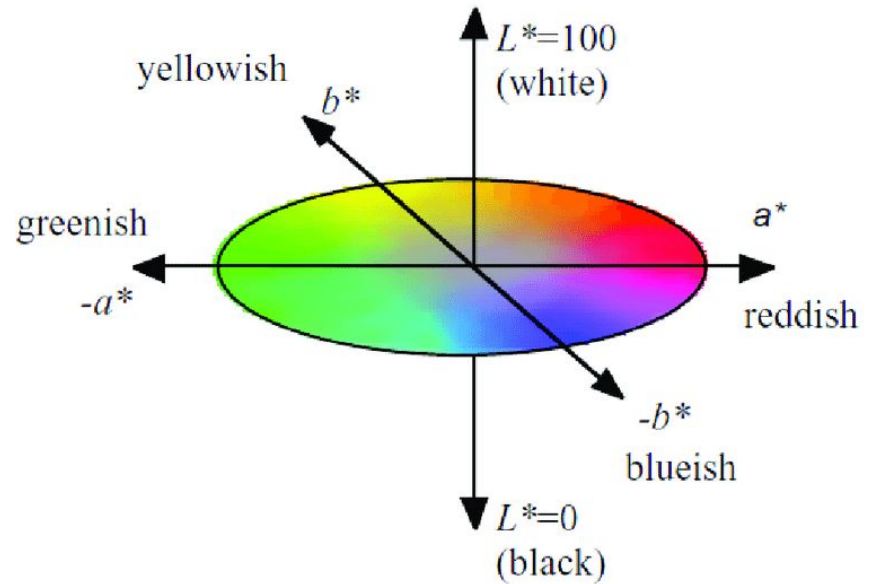
source: https://en.wikipedia.org/wiki/HSL_and_HSV

L^*a^*b

Особенность: однородно воспринимаемое

Значения: (l, a, b)

- l - lightness (светлота)
- a - green-red
- b - blue-yellow



Конвертация

Существуют простые формулы для преобразования из одного цветового пространства в другое.

Пример: RGB -> HSV

$$\begin{aligned} V &\leftarrow \max(R, G, B) \\ S &\leftarrow \begin{cases} \frac{V - \min(R, G, B)}{V} & \text{if } V \neq 0 \\ 0 & \text{otherwise} \end{cases} \\ H &\leftarrow \begin{cases} 60(G - B) / (V - \min(R, G, B)) & \text{if } V = R \\ 120 + 60(B - R) / (V - \min(R, G, B)) & \text{if } V = G \\ 240 + 60(R - G) / (V - \min(R, G, B)) & \text{if } V = B \\ 0 & \text{if } R = G = B \end{cases} \end{aligned}$$

Остальные формулы: https://docs.opencv.org/3.4/de/d25/imgproc_color_conversions.html

Представление изображения

- матрица I размера $H \times W$
 - H - высота
 - W - ширина
- 2х или 3х-мерный массив
- функция двух переменных: $(h,w) \rightarrow I_{h,w}$

Grayscale изображения

Изображение в оттенках серого (grayscale):

пиксель содержит число, обозначающее интенсивность света (яркость)

- вещественное от 0 до 1
- целое от 0 до 255

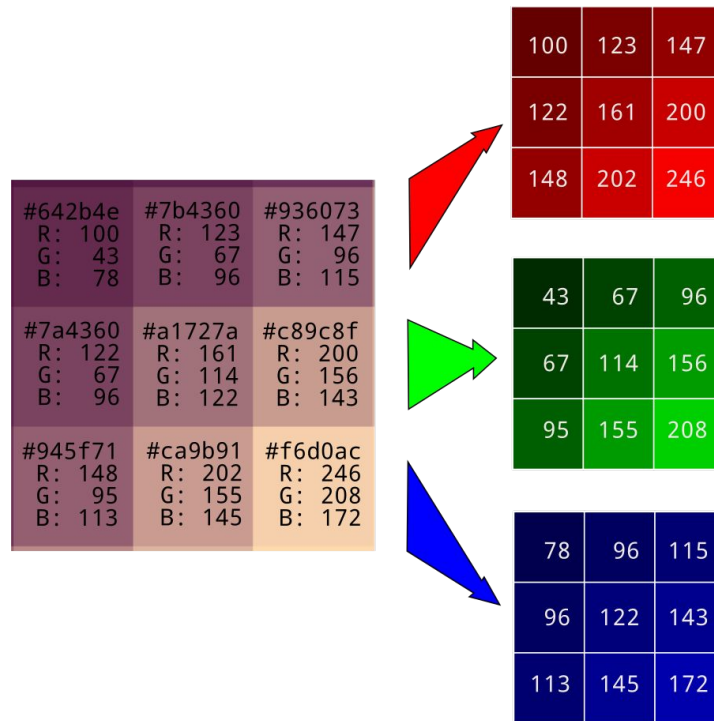


187	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	115	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	125	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Цветные изображения

Цветные изображения: пиксель содержит информацию о цвете
(выраженную в каком-то цветовом пространстве)



Классические методы

Классические методы CV

Существует функция $f: X \rightarrow Y$, где

- X - пространство объектов (изображений)
- Y - пространство ответов (напр., классы)

Задача распознавания: восстановить f

Разумный подход: использовать ML

Классические методы CV

Разумный подход: использовать ML

Проблемы:

1. высокая размерность (вычислительно затратно)
2. зависимость соседних пикселей
3. шумы

Решение:

- обработка изображений
- извлечение признаков

Обработка изображения

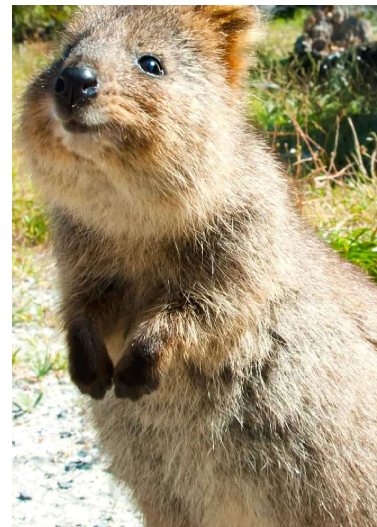
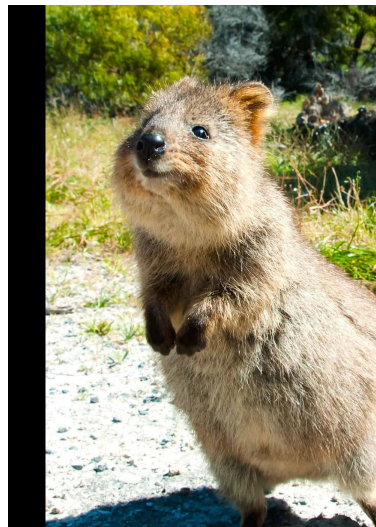
Преобразования изображения

Примеры преобразований:

- геометрические преобразования
- цветовые преобразования
- сглаживание или повышение резкости
- пороговая бинаризация
 - морфологические преобразования

Геометрические преобразования

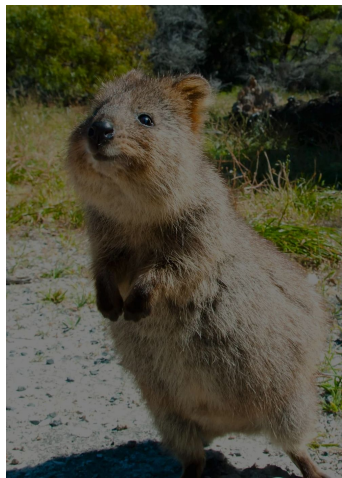
- поворот (rotation)
- трансляция (translation)
- масштабирование (scaling)



source: <https://project-zoo.fandom.com/wiki/Quokka?file=Happy-quokka.jpg> (CC BY-SA)

Цветовые преобразования

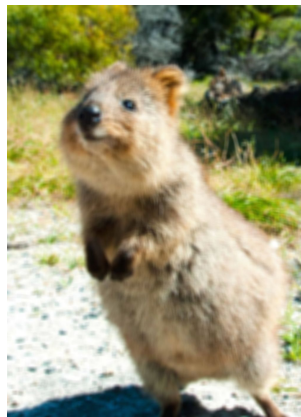
- изменение яркости (brightness adjustment)
- изменение контраста (contrast adjustment)



Сглаживание

Сглаживание (image blurring / image smoothing) - устранение высокочастотных шумов с помощью фильтров.

- усреднение (averaging)
- Гауссов фильтр (gaussian filtering)
- медианный фильтр (median filtering)



Фильтрация

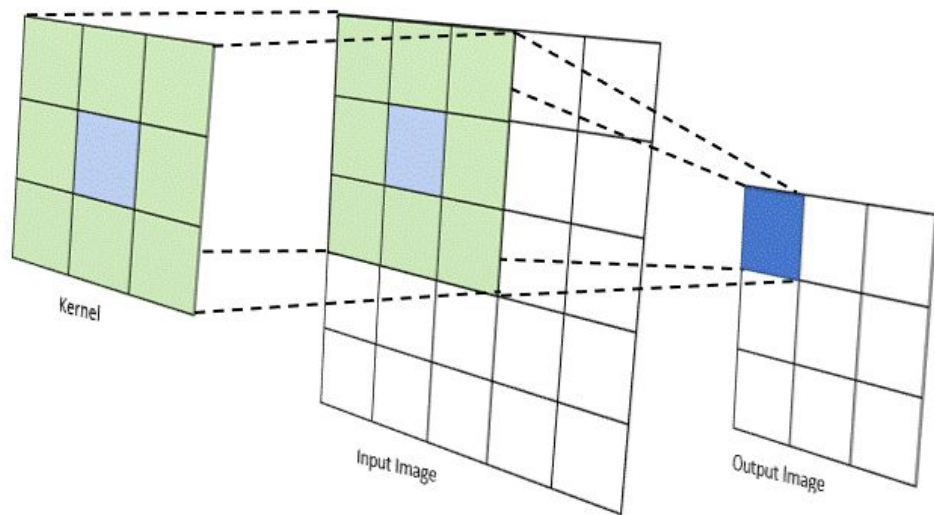
Фильтрация - это процесс обработки изображения, при котором

- берется ядро=фильтр (filter, kernel) - функция над окрестностью пикселя
- ядро применяется к *каждому** пикселю входного изображения, выдавая значение пикселя выходного изображения

* проблема краев решается по-разному: отбрасывание, паддинг, отражение и т.д.

Виды фильтрации по функции ядра:

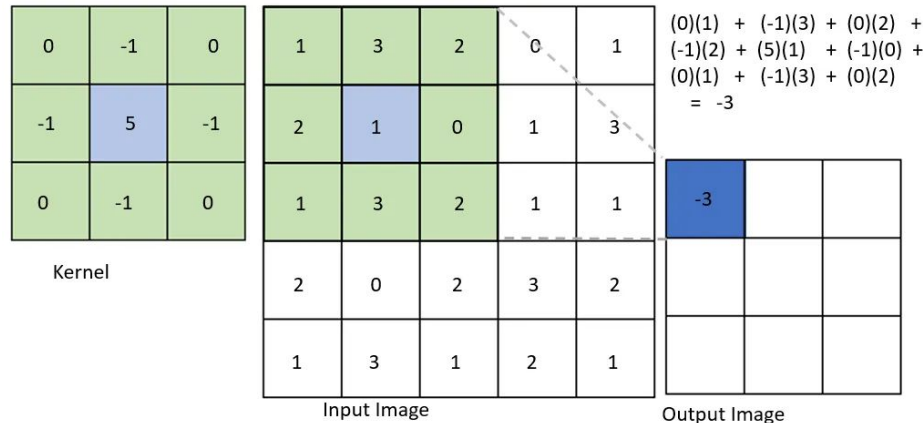
1. Линейная (свёртка)
2. Нелинейная



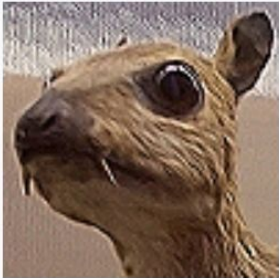
Свёртка

Свёртка: Новое значение пикселя получается как сумма произведений значений ядра и соответствующих пикселей изображения (т.е. произведение Адамара окрестности пикселя и ядра, затем суммирование)

Note: индексация не обращается, в отличие от стандартного определения свёртки в математике



Линейная фильтрация

Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	

Gaussian blur

Гауссово ядро: взвешивание 2-мерной дискретизированной Гауссианой

Параметры:

- размер ядра
- дисперсия σ

Примеры ядер:

1/16

1	2	1
2	4	2
1	2	1

1/273

1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1

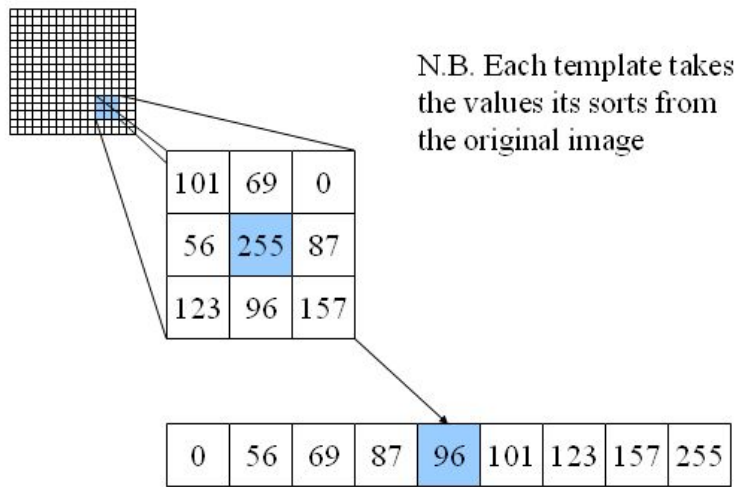
1/1003

0	0	1	2	1	0	0
0	3	13	22	13	3	0
1	13	59	97	59	13	1
2	22	97	159	97	22	2
1	13	59	97	59	13	1
0	3	13	22	13	3	0
0	0	1	2	1	0	0



Нелинейная фильтрация

- Медианный фильтр: функция ядра - медиана используется для борьбы с выбросами (шумом)

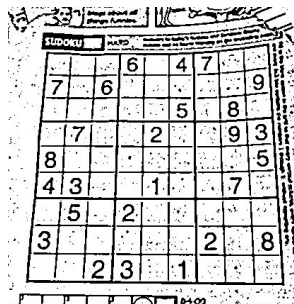
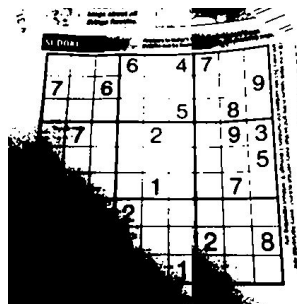
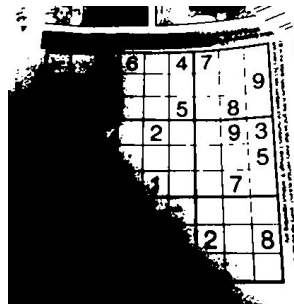


Пороговая бинаризация

Пороговая бинаризация (thresholding):

$$B(x, y) = \mathbb{1}\{I(x, y) > t\}$$

- глобальная - единое значение t на всю картинку
- локальная - разные значения t для разных частей изображения
- адаптивная - значение t зависит от яркости пикселей в окрестности точки
 - mean: t = медиана окрестности.
 - gaussian: t = свертка окрестности с Гауссовым ядром



Морфологические преобразования

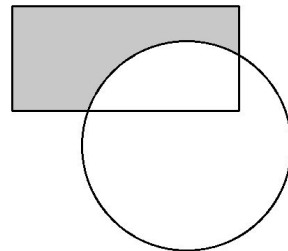
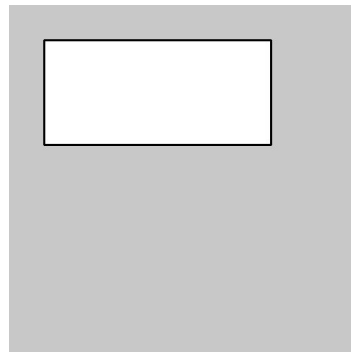
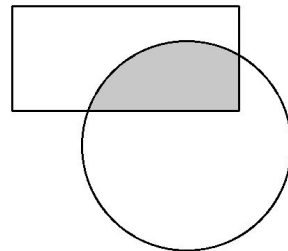
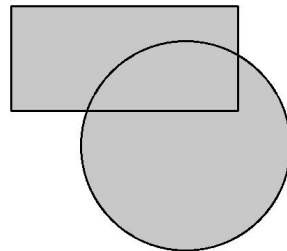
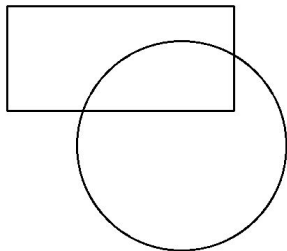
Морфологические преобразования (бинарного изображения) – работают с изображением как с множеством:

- белые пиксели (1) - объект
- черные пиксели (0) - фон

$$B_z = \{b + z | b \in B\}$$

Простейшие морфологические преобразования

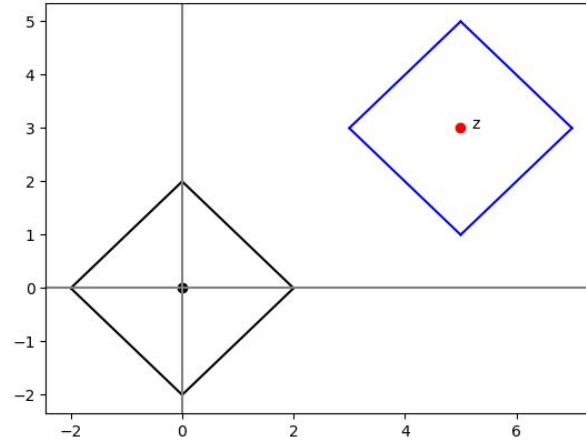
- пересечение
- объединение
- дополнение
- разность



Простейшие морфологические преобразования

Перенос в точку z объекта B :

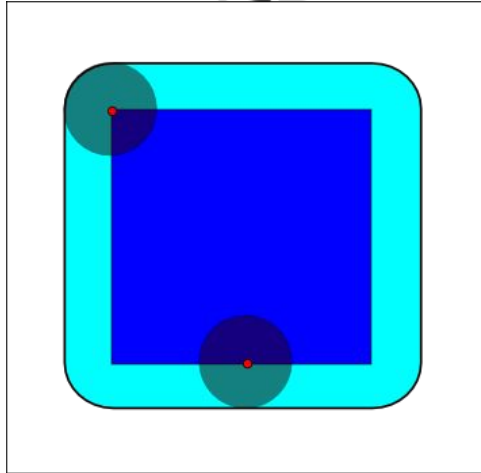
$$B_z = \{b + z | b \in B\}$$



Основные морфологические преобразования

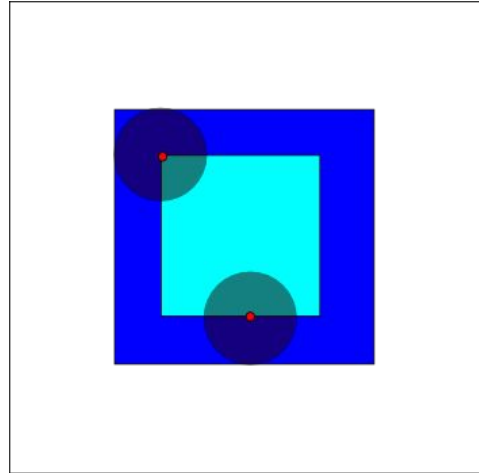
dilation - дилатация, расширение:

$$A \oplus B = \bigcup_{b \in B} A_b$$



erosion - эрозия, сужение:

$$A \ominus B = \{z | B_z \subseteq A\}$$

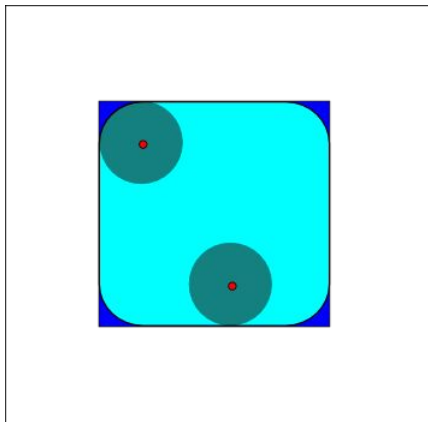


Производные морфологические преобразования

opening - размыкание, открытие:

эрозия -> дилатация

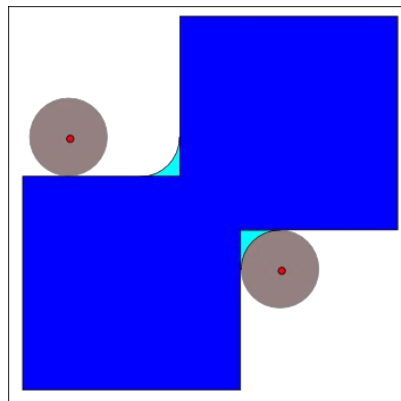
- сглаживает контуры
- убирает небольшие объекты



closing - замыкание, закрытие:

дилатация -> эрозия

- сглаживает контуры объекта
- заполняет небольшие дырки



Признаки изображения

Признаки изображения

Опр. признак изображения (image feature) - это измеримое свойство изображения.

Важно: нужны не любые признаки, а полезные для решаемой задачи.

Критерии полезности:

- инвариантность
- информативность
- компактность

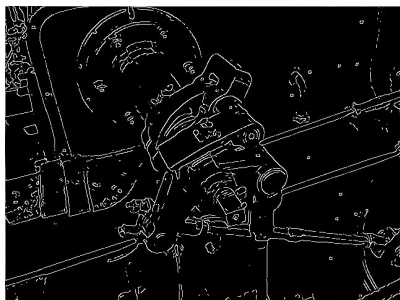
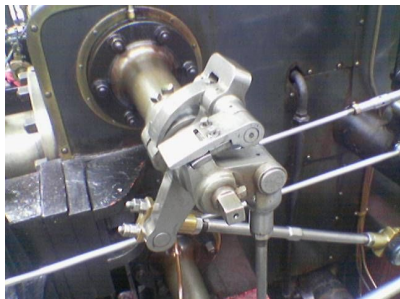


source:

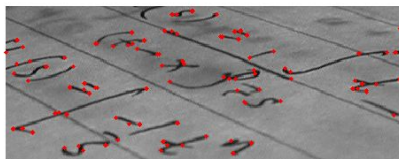
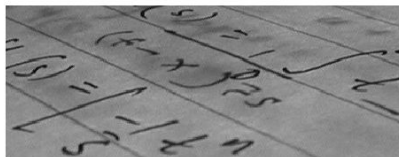
https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_feature2d/py_feature2d_py_feature2d_meaning.html

Примеры признаков

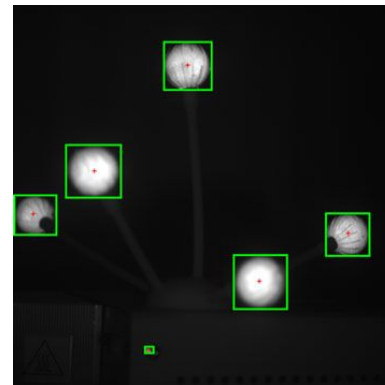
- границы (edges)
- углы (corners) / особые точки (interest points)
- пятна (blobs) / области особых точек (regions of interest points)
- хребты (ridges)



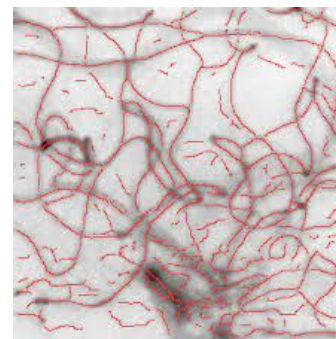
source: https://en.wikipedia.org/wiki/Canny_edge_detector



source: <https://upload.wikimedia.org/wikipedia/commons/5/5f/Corner.png>



source: <https://www.optomotive.com/en/technology/blob-detection/>



source: https://hal.science/hal-00807747/file/Ridges_and_Valleys_Detection_in_Images_using_Difference_of_Rotating_Half_Smoothing_Filters_ACIVS_2011.pdf

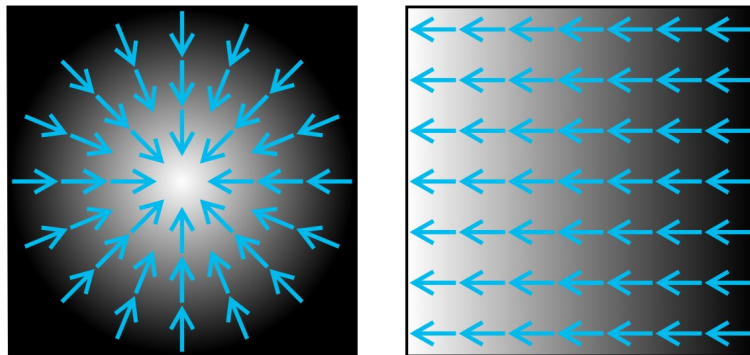
Выделение контуров

Метод – фильтрация оператором градиента:

- **Оператор Собеля (Sobel)** – вычисляет приближённые значения градиента по координатам X и Y , выделяя границы изображения
- **Оператор Лапласа (Laplacian)** – использует вторую производную для обнаружения резких изменений яркости
- **Оператор Кэнни (Canny)** – улучшенный метод, включающий Гауссово размытие, градиент и пороговую фильтрацию

Градиент изображения

Градиент — это вектор, показывающий, насколько сильно и в каком направлении изменяется яркость изображения.



Дискретная производная: 1d

Производная:
$$f'(x) = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

Дискретный аналог:
$$f'(i) = f(i + 1) - f(i)$$

Фильтр: (0, -1, 1)

Более симметричный вариант:

$$\begin{aligned} f(x) &\approx \frac{1}{2\Delta x} ((f(x + \Delta x) - f(x)) + (f(x) - f(x - \Delta x))) = \\ &= \frac{1}{2\Delta x} (f(x + \Delta x) - f(x - \Delta x)) \end{aligned}$$

$$f'(i) = f(i + 1) - f(i - 1)$$

Фильтр: (-1, 0, 1)

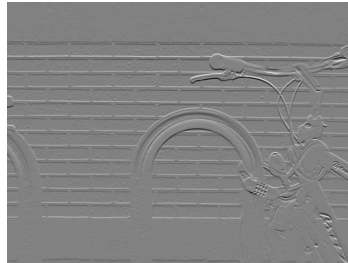
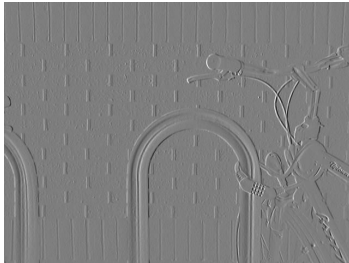
Оператор Собеля

A - исходное изображение, G_x и G_y - массивы производных по x (ширина) и y (высота)

$$\mathbf{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * \mathbf{A} \quad \text{and} \quad \mathbf{G}_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} * \mathbf{A}$$

Из них можно получить для каждого пикселя значение градиента и его направление

$$\mathbf{G} = \sqrt{\mathbf{G}_x^2 + \mathbf{G}_y^2} \quad \Theta = \arctan\left(\frac{\mathbf{G}_y}{\mathbf{G}_x}\right)$$



Оператор Лапласа

Оператор Лапласа: $\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$

$$kernel = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

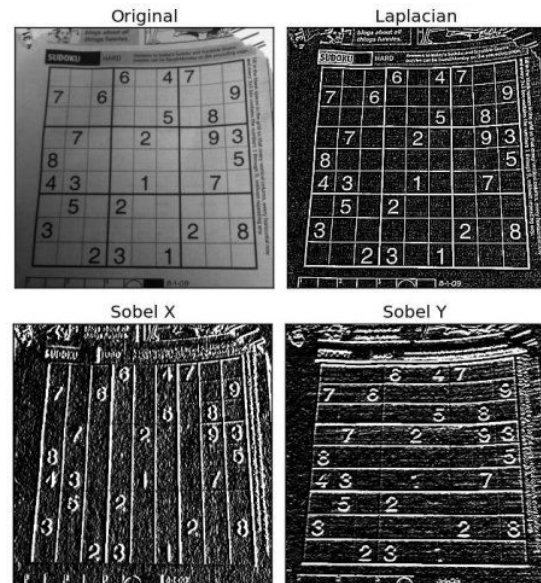
Note: нули - границы

Объяснение:

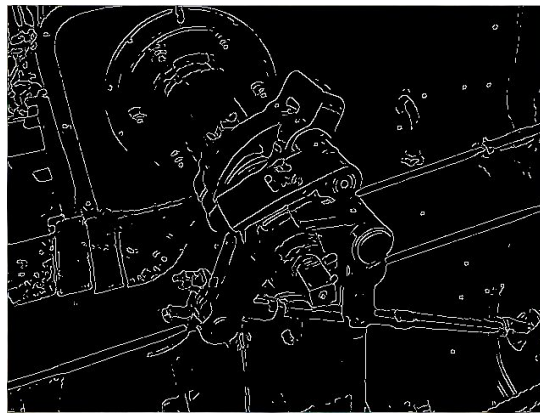
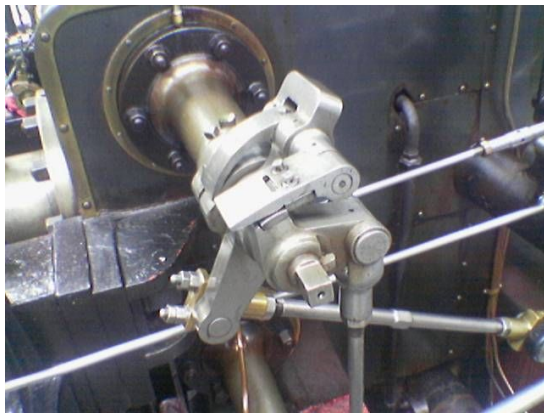
рассмотрим 1d: $\Delta f(x) = f''(x) = \lim_{\Delta x \rightarrow 0} \frac{f'(x + \Delta x) - f'(x)}{\Delta x}$

дискретный аналог: $\Delta f(i) = f'(i + 1) - f'(i) = f(i + 2) + f(i) - 2f(i + 1)$

Фильтр: (1, -2, 1)



Детектор границ Кэнни



source: https://en.wikipedia.org/wiki/Canny_edge_detector

Детектор границ Кэнни

Этапы

1. удалить шум Гауссовым фильтром
2. вычислить величину и направление градиента в каждой точке
3. подавления немаксимумов (non-maximum suppression):
сформировать контуры из точек, являющихся локальными максимумами в направлении градиента
4. трешхолдинг контуров по 2 пороговым значениям $minVal$ и $maxVal$
 - a. $градиент < minVal \Rightarrow$ ненастоящий контур
 - b. $градиент > maxVal \Rightarrow$ настоящий контур
 - c. $minVal < градиент < maxVal$
 - i. точка связана с настоящим контуром $\Rightarrow$ настоящий контур
 - ii. иначе $\Rightarrow$ ненастоящий контур

Детектор границ Кэнни

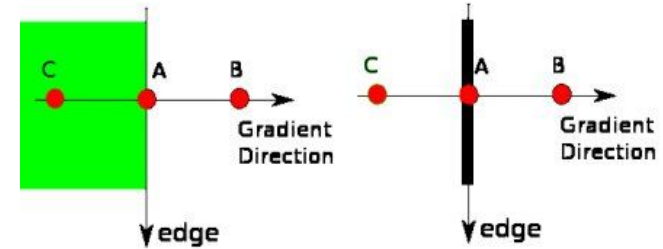
Этапы

1. удалить шум Гауссовым фильтром
2. вычислить величину и направление градиента в каждой точке
3. подавления немаксимумов (non-maximum suppression):
сформировать контуры из точек, являющихся локальными максимумами в направлении градиента
4. трешхолдинг контуров по 2 пороговым значениям $minVal$ и $maxVal$
 - a. $градиент < minVal \Rightarrow$ ненастоящий контур
 - b. $градиент > maxVal \Rightarrow$ настоящий контур
 - c. $minVal < градиент < maxVal$
 - i. точка связана с настоящим контуром $\Rightarrow$ настоящий контур
 - ii. иначе $\Rightarrow$ ненастоящий контур

Детектор границ Кэнни

Этапы

1. удалить шум Гауссовым фильтром
2. вычислить величину и направление градиента в каждой точке
3. подавления немаксимумов градиентов (non-maximum suppression):
сформировать контуры из точек, являющихся локальными максимумами в направлении градиента
4. трешхолдинг контуров по 2 пороговым значениям $minVal$ и $maxVal$
 - a. $градиент < minVal \Rightarrow$ ненастоящий контур
 - b. $градиент > maxVal \Rightarrow$ настоящий контур
 - c. $minVal < градиент < maxVal$
 - i. точка связана с настоящим контуром $\Rightarrow$ настоящий контур
 - ii. иначе $\Rightarrow$ ненастоящий контур



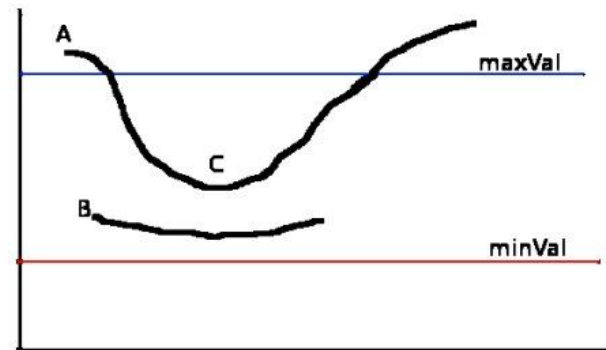
source:

https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_canny/py_canny.html

Детектор границ Кэнни

Этапы

1. удалить шум Гауссовым фильтром
2. вычислить величину и направление градиента в каждой точке
3. подавления немаксимумов (non-maximum suppression):
сформировать контуры из точек, являющихся локальными максимумами в направлении градиента
4. трешхолдинг контуров по 2 пороговым значениям minVal и maxVal
 - a. $\text{градиент} < \text{minVal} \Rightarrow$ ненастоящий контур
 - b. $\text{градиент} > \text{maxVal} \Rightarrow$ настоящий контур
 - c. $\text{minVal} < \text{градиент} < \text{maxVal}$
 - i. точка связана с настоящим контуром $\Rightarrow$ настоящий контур
 - ii. иначе $\Rightarrow$ ненастоящий контур



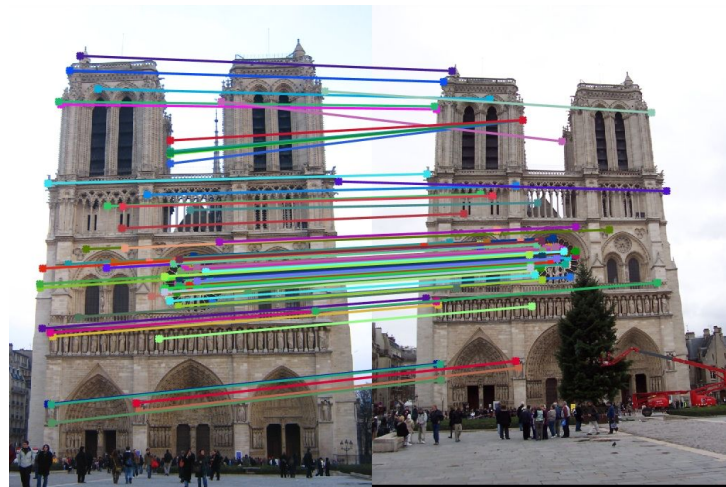
source:

https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_canny/py_canny.html

Распознавание сопоставлением признаков

Компоненты решения задачи распознавания (классификации или детекции):

1. выделение ключевых точек
2. создание дескрипторов (векторов, описывающих локальные характеристики точек)
3. поиск соответствий (matching) между 2 изображениями - путем сравнения дескрипторов ключевых точек этих изображений
4. фильтрация ложных соответствий



Выделение особых точек

Популярные:

1. SIFT (Scale-Invariant Feature Transform)
2. SURF (Speeded-Up Robust Features)
3. ORB (Oriented FAST and Rotated BRIEF)
4. FAST (Features from Accelerated Segment Test)
5. BRIEF (Binary Robust Independent Elementary Features)
6. Harris Corner Detector
7. AKAZE (Accelerated-KAZE)

SIFT

SIFT (Scale-Invariant Feature Transform) — популярный алгоритм для извлечения признаков из изображений.

- + устойчив к масштабированию, повороту, небольшим смещениям и небольшим изменениям освещения.

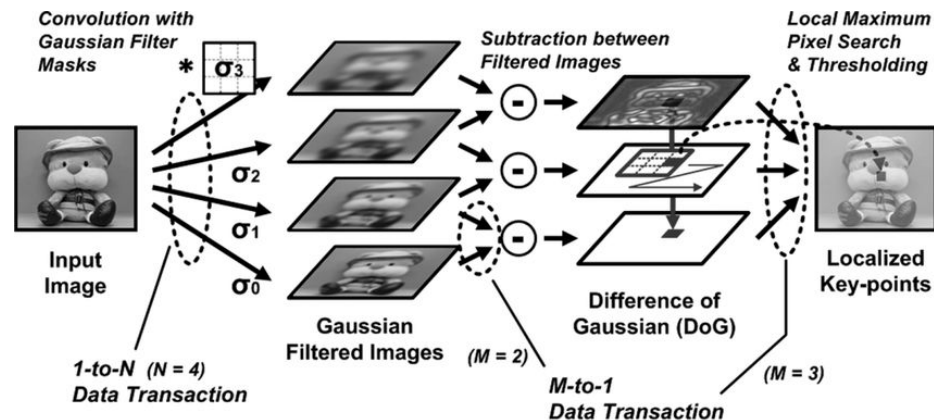
SIFT: компоненты алгоритма

Этап I: детекция ключевых точек

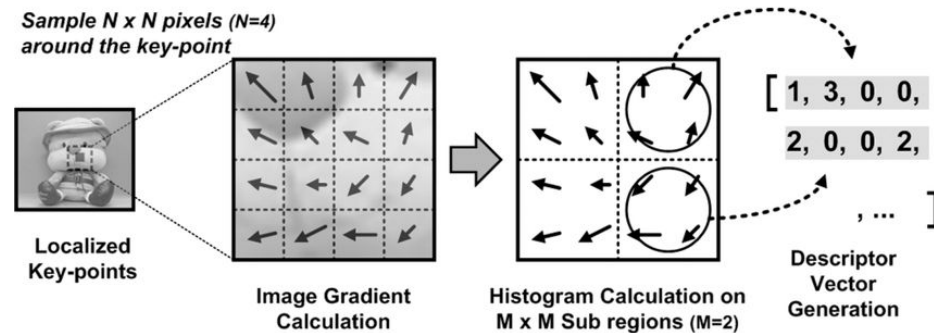
1. построение пирамиды Гауссиан (Gaussian Pyramid): стакаем изображения, размытые все большими sigma
2. подсчет пирамиды разности Гауссиан (Difference of Gaussians, DoG)
3. выявление точек в разностях, обладающих экстремальной интенсивностью в окрестности внутри слоя и соседних слоев
4. отфильтровываем точки с низким контрастом (как шумные) или лежащие у краев (как плохо определенные)

Этап II: построение дескриптора ключевой точки

1. выделяем окрестность и масштабируем
2. вычисляем ориентацию точки на основе градиентов
3. строим гистограмму направлений градиентов в окрестности точки относительно ориентации точки
 - относительно ориентации -> инвариантность к повороту
 - гистограмма направлений (без магнитуды) -> инвариантность к изменению освещения



(a)



(b)

Note: это не точное описание алгоритма

Сопоставление признаков

Примеры дескрипторов:

- цветовые
 - матожидание цвета
 - дисперсия
 - моменты n -го порядка
 - гистограмма яркости
 - гистограммы цветов
- формы
 - длина контура
 - диаметр контура
 - кривизна контура
 - гистограмма ориентированных градиентов
- текстурные
- ...

Сопоставление признаков

Matching:

- Brute-Force Matcher (BFMatcher) - ищет ближайшего соседа перебором
точный & медленный
- FLANN (Fast Library for Approximate Nearest Neighbors) - использует
деревья
менее точный & более быстрый

Фильтрация ложных совпадений:

- crossCheck=True
- Lowe's Ratio Test
- RANSAC (Random Sample Consensus)

Вопросы?